



# **FYDP-DSE D4 template**

FPSEF25X05

***Bachelors of Science in Software Engineering  
(2022-2026)***

**By**

FIZA KHAN BSEF22M011

HUDA EJAZBSEF22M017

ALISHBA UMER BSEF22M018

xx

**Supervised by:**

Dr.Shuja Rehman,  
Assistant Professor

Muhammad Farooq,  
Assistant Professor



**FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY,  
UNIVERSITY OF THE PUNJAB, LAHORE.**

## **D4 Approval Signatures (FAC)**

We, the undersigned members of the assigned Faculty Advisory Committee (FAC), have reviewed the submitted D4 design document. Based on our initial assessment, we acknowledge the group's submission and confirm that the D4 has been evaluated for completeness and alignment with FYDP objectives.

**Group ID: FPSEF25X05**

**Project title: Smart Food Analyzer: Nutrient Profiling from Food Images**

**Primary supervisor:** Dr.Shuja Rehman

**Co-supervisor:** Muhammad Farooq

*Signature:*

*Signature:*

### **Signatures (Faculty Advisory Committee (FAC))**

|                  | <b>FAC member#1</b> | <b>FAC member#2</b> |
|------------------|---------------------|---------------------|
| <b>Name</b>      | Natalia Chauhdary   | Dr.Abdul Khaliq     |
| <b>Signature</b> |                     |                     |



**Head of FYDP Coordination Office:** Dr Natalia Chaudhry

*(Signature)*

## Contents

|                                                         |   |
|---------------------------------------------------------|---|
| 1. Deployment Overview .....                            | 3 |
| 2. Final Live Deployment URL.....                       | 3 |
| 3. Demo Walkthrough Video (if applicable).....          | 3 |
| 4. Deployment Instructions (Manual) .....               | 4 |
| 5. Deployment Architecture Diagram .....                | 4 |
| 6. CI/CD Pipeline Summary (Jenkins/GitHub Actions)..... | 5 |
| 7. Final README.md Snapshot .....                       | 6 |



## 1. Deployment Overview

- The **Smart Food Analyzer** system is deployed as a mobile-based AI application that enables users to upload food images and receive real-time analysis including food detection, portion estimation, and nutritional insights. The deployment integrates a FastAPI backend with deep learning models (YOLO for detection and MiDaS for depth estimation) and a lightweight frontend interface.
- The system is currently deployed in a **local development environment**, with support for future cloud deployment. Special configurations include loading trained model weights, handling image uploads, and API-based communication between frontend and backend.
- **Project Name:** Smart Food Analyzer
- **Version:** v1.0-final
- **Deployment Type:**
  - ☒ Local only

## 2. Final Live Deployment URL

- **Primary Deployment URL:** <http://localhost:8000>
- **Backup / Mirror Deployment:** Not available

## 3. Demo Walkthrough Video (if applicable)

A recorded demonstration that visually showcases the deployed system's features, workflows, and functionality for evaluators who may not access the live deployment directly.

- **YouTube/GooglDriveLink:**  
[https://drive.google.com/file/d/1Zr4S2h6r4Fj4\\_javjWFWFcs6kir9iCbm/view?usp=drive\\_link](https://drive.google.com/file/d/1Zr4S2h6r4Fj4_javjWFWFcs6kir9iCbm/view?usp=drive_link)



- **Duration:** ~3–5 minutes
- **Narration:** No
- **Co**
  - ☒ Image Upload
  - ☒ Food Detection Output
  - ☒ Nutritional Analysis
  - ☐ User Login (optional)

## 4. Deployment Instructions (Manual)

### a. Prerequisites

- Python 3.10+
- PyTorch (for AI models)
- Git
- pip
- Virtual Environment (recommended)
- 

### b. Setup Steps

#### Step 1: Clone repository

```
git clone https://github.com/foodanalyzer/SmartFoodAnalyzer
```

```
cd smart-food-analyzer
```

#### Step 2: Create virtual environment

```
python -m venv venv venv\Scripts\activate # (Windows)
```

#### Step 3: Install dependencies

```
pip install -r requirements.txt
```



## **Step 4: Place model weights**

**Add YOLO and MiDaS models inside /models directory**

## **Step 5: Run backend server**

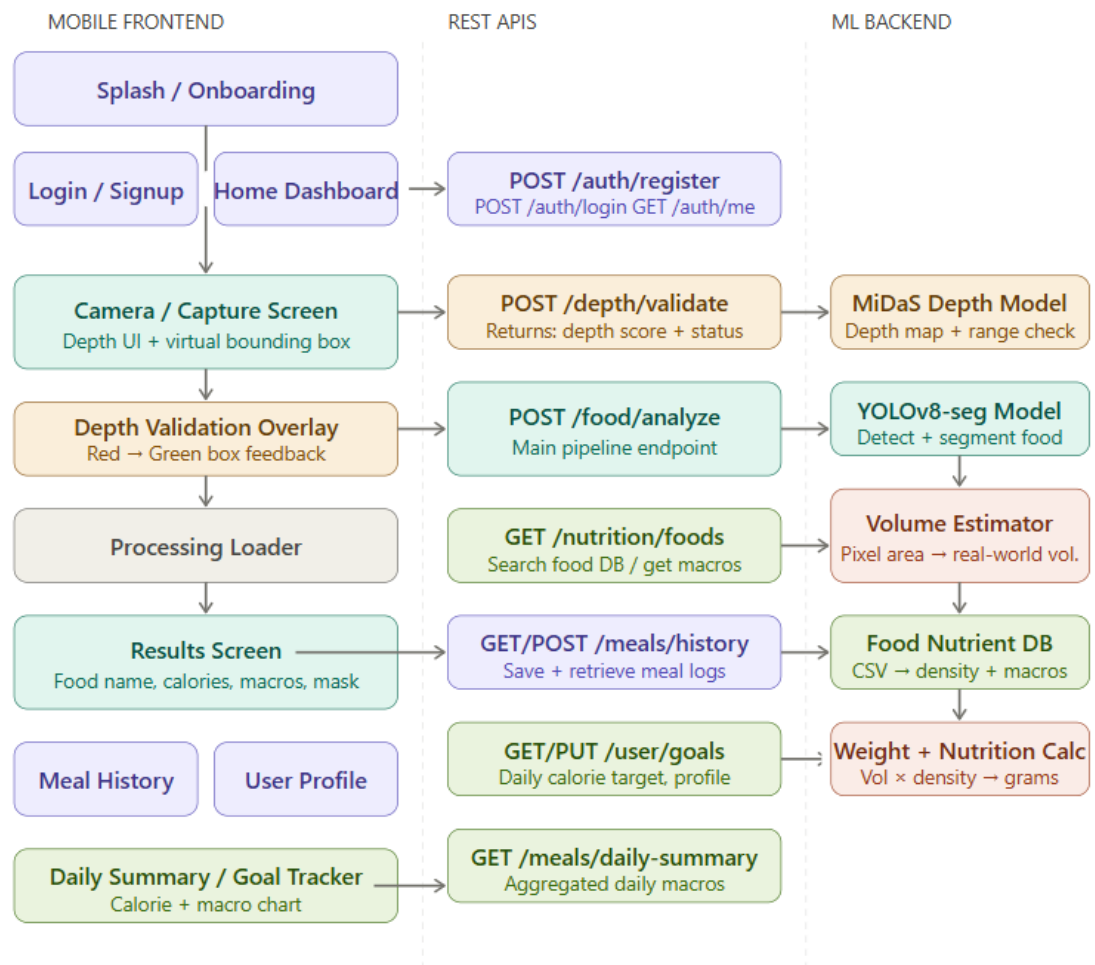
uvicorn main:app --reload

## **Step 6: Open application**

<http://localhost:8000>



## 5. Deployment Architecture Diagram



## 6. CI/CD Pipeline Summary (Jenkins/GitHub Actions)

- **Tool Used:** ☒ GitHub Actions ☐ Jenkins
- **Trigger Type:**
  - ☒ Push to main
  - ☒ Pull Request Merged
- **Pipeline Stages:**
  - Code Checkout
  - Dependency Installation
  - Basic Testing
  - Build Validation



- Deployment

## 7. Final README.md Snapshot

### SmartFoodAnalyzer

AI-powered food recognition and nutritional analysis system – point your camera at food, get instant calories, macros, and health insights.

#### Overview

SmartFoodAnalyzer uses a multi-model AI pipeline to:

Validate camera distance in real-time using MiDaS  
monocular depth estimation

Detect and segment food using YOLOv8-Seg

Estimate portion weight from 3D volume  
reconstruction

Return nutritional breakdown – calories, protein,  
carbs, fat, fiber





## AI Pipeline

Camera Frame

|



[MiDaS Depth Validation] → too near / too far →  
reject

|

valid



[YOLOv8-Seg Detection] → food class +  
segmentation mask

|



[Mask Area Calculation] → pixel area of food  
region

|



[3D Volume Estimation] → depth map × mask area  
× calibration

|





[Weight Estimation] → volume × density (from nutrition DB)

|



[Nutrition Mapping] → calories, protein, carbs, fat, fiber

Getting Started

Prerequisites

Python 3.10+

PostgreSQL 14+

Redis (for Celery task queue)

Installation & Setup

## # 1. Clone the repository

git clone <https://github.com/foodanalyzer.git>

cd SmartFoodAnalyzer



## # 2. Create and activate virtual environment

```
python -m venv venv  
source venv/bin/activate          # Linux/macOS  
venv\Scripts\activate            # Windows
```

## # 3. Install dependencies

```
pip install -r requirements.txt
```

## # 4. Set up environment variables

```
cp .env.example .env
```

# Edit .env with your DB credentials, JWT secret,  
and model paths

## # 5. Initialize the database

```
python src/db/init_db.py
```

## # 6. Load the nutrition CSV into PostgreSQL

```
python src/nutrition/load_csv.py
```

## # 7. Start the FastAPI server



```
uvicorn src.main:app --reload --host 0.0.0.0 --port  
8000
```

## # 8. Start Celery worker (in a separate terminal)

```
celery -A src.tasks worker --loglevel=info
```

## Environment Variables (.env)

```
envDATABASE_URL=postgresql://user:password@localhost  
:5432/smartfood
```

```
REDIS_URL=redis://localhost:6379/0
```

```
JWT_SECRET=your_secret_key_here
```

```
JWT_EXPIRE_MINUTES=1440
```

```
MODEL_PATH=model/yolov8_food_seg.pt
```

```
MIDAS_MODEL=model/midas_v21_small.pt
```

```
IMAGE_STORAGE=local # or "s3"
```

```
AWS_BUCKET= # only if
```

```
IMAGE_STORAGE=s3
```

Tech Stack



| Layer         | Technology                              |
|---------------|-----------------------------------------|
| Backend       | APIFastAPI(Python)                      |
| ML Models     | YOLOv8-Seg, MiDaS                       |
| Database      | PostgreSQL + SQLAlchemy                 |
| Task          | QueueCelery + Redis                     |
| Auth          | JWT + bcrypt                            |
| Model Serving | PyTorch / ONNX                          |
| Nutrition DB  | CSV → PostgreSQL (pg_trgm fuzzy search) |
| Image Storage | Local filesystem / AWS S3               |
| Mobile Client | React Native (Expo)                     |